

SIMATS ENGINEERING

Department of Computer Science and Engineering



CSA15 Cloud Computing and Big Data Analytics

CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense

Editable student template for application-based cloud virtualization and SDDC/cloud operations defense

Assessment Metadata

Course Code	CSA1512
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense
Submission Type	Individual digital case analysis and infrastructure defense based on the same capstone title used in CO1, CO2 AT1 and CO2 AT2
Faculty	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Submission Mode	Completed DOCX template and GitHub repository link through Google Classroom

How This Template Works

- Use the same capstone title used in CO1, CO2 AT1 and CO2 AT2.
- Do not recalculate all VM sizing or repeat all configuration screenshots. Summarize the final AT1/AT2 evidence and use it for design defense.
- Answer all five questions with application-oriented reasoning connected to the capstone product.
- Include sustainable cloud operation decisions wherever relevant: right-sized resources, log retention, storage lifecycle, scheduled shutdown, API-call reduction and VM/container/serverless comparison.
- Paste the final topology diagram in the given response area. The sample diagram is only a structure; redraw it for the student's own product.
- Upload this completed document and supporting evidence to GitHub, then submit the repository link through Google Classroom.

Assessment Focus

AT1 answered what VM is required. AT2 showed configuration or simulation evidence. AT3 asks whether the student can defend the complete infrastructure design under application, security, operations and failure conditions.

Student and Capstone Details

Student Name	D.S.THARUNIYA PRIYA
Register Number	192511317
Class / Slot	Slot B
Capstone Title	PulsePay — Fake Payment Screenshot Detection Using Cloud Computing

CO1 Evidence Link	https://drive.google.com/file/d/1E7tn6EWhS0_Z5pj5TPEspjxWzCOxPQbl/view
CO2 AT1 VM Recommendation	vCPU: 2-4 RAM:12 GB Storage: 40 GB VM Class: API backend + separately-scaled AI/Python service
CO2 AT2 Evidence Link	https://drive.google.com/file/d/1h3p0xD88PV-Bv2EnVxLXFH0K-m053xsh/view
GitHub Repository Link	https://github.com/tharu-ui/Cloud-Computing-and-BDA
Submission Date	31-07-2026

Evidence Carry-Forward Summary

Evidence Source	Student Entry	How It Supports AT3 Defense
CO1 concept map / presentation	PulsePay cloud concept map linking Cloud Ecosystem, API Role, Service Models, Elasticity and PulsePay Modules (OCR/ELA/ML Scoring)	Establishes the original service selection that this AT3 design defends and elaborates on
CO2 AT1 sizing workbook	2-4 vCPU / 12GB RAM / 40GB disk, based on 1,000 registered merchants, ~200 concurrent peak, 3.33 RPS	Provides the quantitative basis for the VM/service sizing
VM calibration sheet	CPU score 8 (API + Auth + DB + Background jobs + Analytics + AI module), RAM 12GB with 25% buffer	Justifies why compute is split between a light API layer and a heavier AI/ELA service
CO2 AT2 practical evidence	Local VM (Alpine, VirtualBox), Docker (NGINX container), Azure VM (Ubuntu 24.04, B2s_v2) — all boot/verify/secure/cleanup evidence captured	Demonstrates hands-on understanding of VM vs. container vs. cloud VM tradeoffs, directly informing the lifecycle and design-alternative reasoning
Capstone architecture decision	Serverless-first hybrid: Cloud Functions/Cloud Run for compute, Firestore for database, Cloud Storage for images	Forms the deployment topology defended i

Q1: Virtualization Value and Capstone Cloud Deployment Summary

Explain your capstone product, expected workload, cloud need and deployment context. Analyze how virtualization improves cost efficiency, resource utilization, workload isolation, infrastructure flexibility and cloud deployment readiness for your product.

Design Point	Student Response
Capstone product and users	PulsePay is a fraud-detection tool that verifies whether a UPI payment screenshot is genuine or edited. Users are small merchants, online sellers, and gig/delivery workers who receive screenshots as informal proof of payment.
Expected workload	~1,000 registered merchants, ~200 concurrent users at peak, ~3.33 requests/second, ~1,500 verification requests/day — short, bursty

	requests rather than sustained load
Cloud need	On-demand compute for OCR and image forensics, managed storage for screenshots and records, and elastic scaling during festival/sale-season spikes without maintaining idle infrastructure
Virtualization value	Virtualization (and its evolution into serverless/managed compute) lets PulsePay provision compute per verification request instead of running a dedicated server continuously, matching its spiky, low-average-load traffic pattern
Cost and utilization benefit	Pay-per-invocation billing (Cloud Functions) means merchants generate cost only when actually verifying a screenshot — no idle server cost during the many hours with zero traffic
Isolation and flexibility benefit	Each verification request runs in its own isolated function/container instance, so a slow or failing OCR call for one merchant cannot affect another merchant's request; compute can also be moved from function to container (Cloud Run) for the AI module without redesigning the rest of the system
Case Analysis Response: Write 8-12 technical lines connecting virtualization value to your capstone product. PulsePay's workload is fundamentally bursty: a small merchant checks a handful of screenshots per day, with demand concentrated around sales and festival periods. Virtualization — and its more modern serverless form — is what makes this economically viable, since it allows compute to be provisioned only for the seconds a verification actually takes, rather than running a server 24/7 for occasional use. This directly improves cost efficiency, since billing is tied to actual OCR/ELA processing time rather than fixed server-hours. Resource utilization improves because the same underlying compute pool is shared across all merchants instead of each one needing dedicated infrastructure. Isolation matters because a single merchant's malformed upload or a slow OCR call should never affect another merchant's verification request, and function/container-level isolation guarantees this. Flexibility is demonstrated in the CO2 AT2 evidence itself: the same NGINX service was run as a full VM, a Docker container, and an Azure cloud VM, showing that the underlying architecture can shift compute models (VM to container to serverless) as the product matures, without needing to redesign the application logic. This flexibility is exactly why the AI/ELA module is planned as a container rather than a fixed VM — deployment readiness improves because scaling up (more merchants) or down (off-season) requires no manual server provisioning at all.	

Q2: Rapid Provisioning, Testing and Virtual Machine Lifecycle Case

Using your CO2 AT1 VM recommendation and CO2 AT2 practical evidence, explain how virtual machines, VM images, templates, snapshots, clones or containers support rapid provisioning, testing, deployment, rollback and maintenance for your capstone product.

Lifecycle Element	AT1/AT2 Evidence Used	Capstone Use / Justification
VM image or base environment	Alpine Linux (local VM) and Ubuntu 24.04 LTS (Azure VM) both verified via <code>uname -a/free -h/df -h</code>	Confirms a lightweight, reproducible base image can be stood up quickly for testing PulsePay's backend logic before deploying to serverless production
Template or repeatable setup	Azure VM created with a defined config (B2s_v2, Ubuntu 24.04, SSH-key auth, NSG rules) that can be	The same VM configuration table can be reused to spin up a fresh test environment for OCR/ELA experiments without

	redeployed identically	reconfiguring from scratch
Snapshot / rollback	Snapshot before-update-or-demo taken on the local VM before any changes	Mirrors how a Firestore export or model-version snapshot would protect PulsePay's verification records before a schema or model change
Clone / testing copy	Not attempted in CO2 AT2	Confirms the decision to run PulsePay's AI/ELA module as a container (Cloud Run) rather than a VM, since containers start faster and use resources only while active
Container or VM decision	Docker NGINX container (8s startup-class) compared directly against the Alpine VM (much slower boot, fixed 2GB reservation)	Confirms the decision to run PulsePay's AI/ELA module as a container (Cloud Run) rather than a VM, since containers start faster and use resources only while active
Maintenance and update plan	Azure VM's package update flow (apt update, apt install nginx) observed directly	Shows how OS/package updates would be applied to any VM-based component; production, however, avoids this entirely by using managed services (Firestore, Cloud Storage) that don't need OS patching

Case Analysis Response: Explain how lifecycle practices reduce deployment delay and risk.

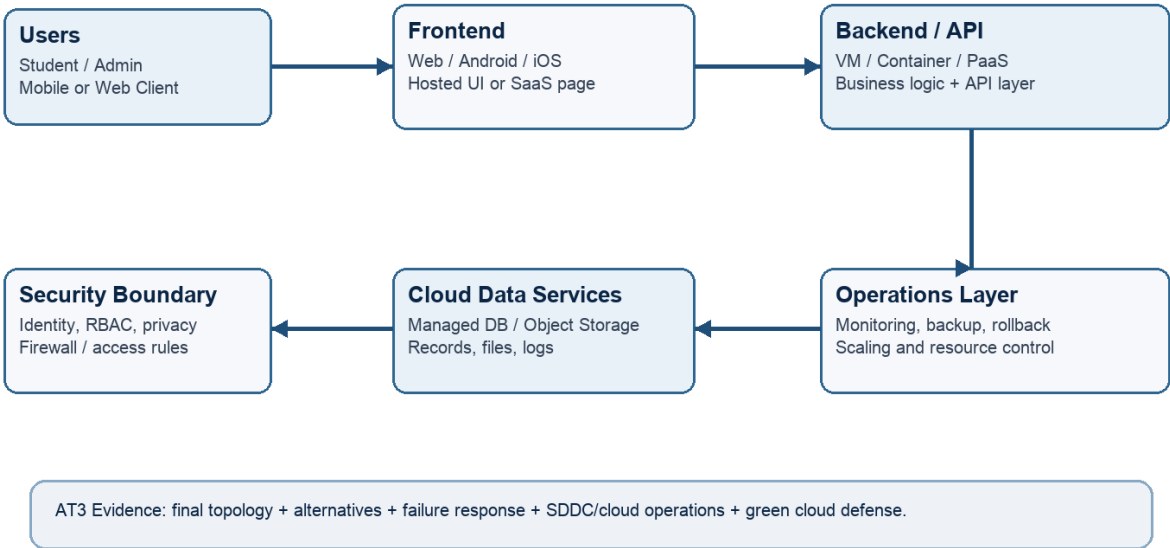
Rapid provisioning matters for PulsePay because the AI/ELA module needs a stable, testable environment before every model or logic change reaches production. The CO2 AT2 evidence shows that a fresh VM (local or cloud) can be stood up from a known image in minutes and verified with a fixed set of commands, meaning a broken deployment can be diagnosed quickly by comparing against that known-good baseline. Snapshots reduce risk by giving a rollback point immediately before any risky change — the same logic applies directly to PulsePay's Firestore records and model versions, where a snapshot/export before a schema change prevents permanent data loss if a migration fails. The VM-vs-container comparison (Alpine VM vs. Docker NGINX) demonstrated concretely that containers start in under a second versus the VM's much longer boot, directly justifying why PulsePay's bursty, per-request AI workload is best served by containers/serverless functions rather than a persistently running VM. Together, these lifecycle practices — repeatable images, snapshots, and the container-first decision — reduce both deployment delay (faster to spin up test environments) and risk (rollback points exist before every risky change).

Q3: Final Cloud Deployment Topology and SOA/API Service Integration Case

Draw and explain the final deployment topology for your capstone product. Show frontend, backend/API, database, storage, authentication, notification, analytics and any VM/container/managed cloud service components. Explain how SOA/API-based integration connects these modules and supports modular cloud service design.

Capstone Cloud Deployment Topology - Sample Structure

Students should redraw this with their own modules, services, VM/container choices and security boundaries.



Architecture Component	Technology / Service	VM / Container / Managed Service / SaaS	Integration Method
Frontend	React web app	Managed Service (Firebase Hosting)	Static assets served via CDN; calls backend via REST API
Backend/API	Cloud Functions + API Gateway	Serverless Function	Exposes REST endpoints consumed by the frontend over HTTPS
Database	Firestore	Managed Service (SaaS)	Accessed via authenticated SDK/API calls from Cloud Functions
Storage	Cloud Storage	Managed Service (SaaS)	Screenshots uploaded via signed URLs, referenced by Firestore records
Authentication	Firebase Authentication	Managed Service (SaaS)	Issues tokens validated by API Gateway on every request
Notification	Cloud Functions + Pub/Sub	Serverless Function	Triggered on high-risk score, sends alert via event-driven function

Analytics / AI module	Cloud Vision API (OCR) + Cloud Run (ELA/ML scoring)	Container (Cloud Run) + Managed API	Cloud Function calls Vision API synchronously, then invokes a Cloud Run container for heavier ELA/scoring processing
-----------------------	---	-------------------------------------	--

Paste or draw the final capstone deployment topology diagram here.

PulsePay Cloud Deployment Topology

```
graph LR; Merchants[Merchants<br/>Small sellers / gig workers<br/>Web or mobile client] --> Frontend[Frontend<br/>React web app<br/>Firebase Hosting (CDN)]; Frontend --> Backend[Backend / API<br/>Cloud Functions + API Gateway<br/>OCR (Vision API) + ELA + ML scoring]; Backend --> Ops[Operations layer<br/>Cloud Monitoring, backup / rollback<br/>Auto-scaling, fraud analytics dashboard]; Ops --> Data[Cloud data services<br/>Firestore (records)<br/>Cloud Storage (screenshots)]; Data --> Security[Security boundary<br/>Firebase Auth, IAM/RBAC<br/>Encryption at rest / in transit];
```

SOA/API Integration Explanation: Explain how services communicate and why modular service design is useful.

Each PulsePay module exposes a narrow, well-defined interface rather than sharing internal state directly: the frontend never touches Firestore or Cloud Storage directly, it only calls the backend's REST API, which then handles authentication, OCR, and forensic scoring internally. This SOA-style separation means each service — auth, storage, OCR, scoring, notifications — can be updated, scaled, or even replaced independently without breaking the others, as long as the API contract stays the same. Modular design is particularly useful here because the AI/ELA module is the most likely component to change over time (new fraud patterns, better models), and isolating it behind its own API/service boundary means that evolution doesn't require redeploying the frontend or database layer.

Q4: Design Alternatives, Identity, Access and Federated Cloud Privacy Case

Compare at least two deployment alternatives for your capstone product, such as single VM, multi-tier VM, containerized deployment, managed backend, federated cloud or hybrid cloud model. Justify the selected design by analyzing identity management, role-based access control, federated access, data sharing, privacy protection and security risks.

Deployment Alternative	Advantages	Limitations / Risks	Identity and Privacy Impact
Alternative 1	Simple to set up and reason about; one place to manage logs and config	Doesn't scale well under bursty traffic; wastes cost running 24/7 for occasional use; a single point of failure for the whole backend	All merchant data and screenshots would sit behind one access boundary, making a single compromised credential a bigger risk
Alternative 2	Scales automatically with traffic; pay-per-use cost; each service (auth, storage, compute) has	Slightly more complex to design and monitor across multiple managed services; cold-start	Access is naturally compartmentalized — a leaked API key for one function doesn't automatically expose the database or storage bucket, since each service has its

	its own access boundary	latency on first request	own IAM policy
Selected design	Serverless + managed services (Alternative 2)	accepted as reasonable tradeoffs given PulsePay's low, bursty traffic and low ops capacity	access compartmentalization is the deciding factor for a product handling partial financial data
Security Area		Student Decision	
User roles		Merchant (uploads screenshot, views verdict); Admin (monitors flagged cases and analytics)	
Authentication method		Firebase Authentication, token-based (no passwords stored by PulsePay itself)	
Role-based access control		IAM roles scoped per Cloud Function — the OCR/scoring function can read/write only its own Firestore collections and Storage bucket, not admin-level resources	
Federated access / partner access		PulsePay does not yet integrate with external bank/NPCI APIs; planned as a future federated integration for settlement verification	
Sensitive data handled		Payment screenshots that may contain partial UPI IDs, transaction amounts, and timestamps	
Privacy protection decision		Encryption at rest (Cloud KMS) and in transit (TLS); raw screenshots auto-deleted after a retention window; only hashed references retained for analytics	
Case Analysis Response: Defend why the selected design is better for identity, access and privacy.			
The serverless + managed-services design is better for identity, access and privacy specifically because it avoids a single broad access boundary. In a single-VM design, one compromised SSH key or misconfigured firewall rule could expose every merchant's screenshots and verification history at once — exactly the failure mode observed conceptually in CO2 AT2, where the Azure VM's temporarily-open port 80 was the only inbound surface beyond SSH, and even that was closed immediately after testing. With Firebase Authentication issuing per-request tokens and IAM roles scoped per Cloud Function, a leaked credential for the OCR function does not automatically grant access to the Firestore database or the Cloud Storage bucket — each service enforces its own access policy independently. This compartmentalization matters most for PulsePay because the data involved (partial UPI IDs, transaction amounts) is directly financially sensitive, so limiting the blast radius of any single credential leak is a higher priority than the operational simplicity a single VM would offer.			

Q5: Software-Defined Datacenter and Cloud Operations Defense Case

Analyze how your infrastructure will respond to operational and failure scenarios such as VM failure, database growth, traffic spike, credential leakage, service outage, resource contention or cost increase. Explain how SDDC concepts and cloud operations practices such as automated provisioning, software-defined networking, software-defined storage, monitoring, logging, backup, rollback, scaling, resource control and maintenance improve the final infrastructure design. Include Green Computing decisions such as resource right-sizing, storage lifecycle control, log retention, API-call reduction, scheduled shutdown, VM/container/serverless comparison and carbon/energy proxy indicators where relevant.

Operational / Failure Scenario	Expected Impact	SDDC / Cloud Operations Response	Evidence or Control
--------------------------------	-----------------	----------------------------------	---------------------

VM failure	A single VM going down would take the whole backend offline	Serverless functions have no single VM to fail — Cloud Functions automatically reschedules failed invocations on healthy infrastructure	Demonstrated conceptually via CO2 AT2's Docker container restart behavior vs. a VM needing manual intervention
Database growth	Slower queries and higher storage cost as verification records accumulate	Firestore auto-scales storage and indexing; a data-retention policy archives/deletes old records beyond a set window	Retention window already planned in CO2 AT1 storage sizing
Traffic spike	Increased latency or dropped requests during festival/sale season	Cloud Functions auto-scale concurrency; Firestore's serverless capacity absorbs spikes without manual provisioning	AT1 traffic model already assumes a 2x peak factor
Credential leakage	A leaked API key could expose merchant data or run up cost	IAM least-privilege roles per function; key rotation; Firebase Auth token expiry limits leaked-token exposure window	SSH-key-only auth pattern demonstrated in CO2 AT2 Azure VM
Service outage	Merchants unable to verify screenshots during downtime	Multi-service architecture means a Cloud Storage outage doesn't necessarily take down authentication or the frontend; status page and retry logic planned	SSH-key-only auth pattern demonstrated in CO2 AT2 Azure VM
Resource contention	High-traffic merchants slow down verification for others	Function-level isolation ensures each invocation runs independently; no shared VM resource contention like CPU/RAM starvation	Demonstrated by Docker's per-container isolation in AT2
Cost increase	Unexpected billing from runaway resource usage	Auto-shutdown/budget alerts (as configured on the AT2 Azure VM), quotas on Cloud Functions concurrency, and monitoring dashboards	Azure VM Auto-shutdown (11 PM IST) set up as a direct precedent
Operational Readiness Area		Student Plan	
Automated provisioning		Cloud Functions and Firestore provision automatically per request — no manual server setup needed for scaling	
Software-defined networking		API Gateway and IAM policies define access rules in configuration rather than physical network changes, mirroring the NSG port-rule approach used in CO2 AT2	
Software-defined storage		Cloud Storage lifecycle rules automatically tier/delete objects based on age, without manual disk management	

Monitoring and logging	Cloud Monitoring + Cloud Logging track function errors, latency, and fraud-detection outcomes		
Backup and rollback	Firestore exports before schema changes; Cloud Function versioning allows rollback to a previous deployed version		
Scaling and elasticity	Cloud Functions concurrency auto-scales with request volume; no pre-provisioned capacity required		
Resource control and maintenance	Managed services eliminate OS patching; only the AI/ELA container image needs periodic dependency updates		
Green Computing and sustainability control	Serverless-first design means zero idle compute between requests, directly reducing energy use compared to an always-on VM		
Green Cloud Decision		Student Justification	
VM right-sizing from CO2 AT1	Sized at 2-4 vCPU/12GB rather than defaulting to a larger VM, avoiding over-provisioned idle capacity		
VM vs container vs serverless choice	Serverless for the API layer, container only for the heavier AI/ELA module — avoids running a full VM 24/7 for infrequent requests		
Scheduled shutdown / idle-resource control	Demonstrated directly in CO2 AT2 via Azure VM Auto-shutdown (11 PM IST); production serverless functions have no idle cost by design		
Log and file retention policy	Verification logs and screenshots retained only for a defined window, then deleted, reducing storage footprint over time		
Storage lifecycle and backup cleanup	Verification logs and screenshots retained only for a defined window, then deleted, reducing storage footprint over time		
API-call or polling reduction	Verification logs and screenshots retained only for a defined window, then deleted, reducing storage footprint over time		
Carbon/energy proxy indicator to track	Verification logs and screenshots retained only for a defined window, then deleted, reducing storage footprint over time		
Final Infrastructure Defense Statement: State whether the design is ready for prototype, pilot or production. Defend reliability, security, SDDC/cloud operations and Green Computing improvements required next.			
This design is ready for prototype and early pilot stages, but not yet production. Reliability is reasonable for a pilot: serverless compute removes single-VM failure risk, and Firestore/Cloud Storage handle their own redundancy. Security is addressed at a basic level — IAM least-privilege roles, token-based auth, and encryption at rest/in transit are planned, though live bank/NPCI verification integration (a federated access scenario) is not yet built and would need its own access review before production. SDDC/cloud-operations maturity is currently conceptual for most controls (monitoring, backup, scaling) since CO2 AT2's practical evidence covered VM/container fundamentals rather than the full production stack; the next step is implementing actual Cloud Monitoring dashboards and Firestore backup automation rather than just planning them. Green Computing is a genuine strength of this design: the serverless-first approach avoids idle compute by construction, and the CO2 AT2 evidence (Auto-shutdown on the Azure VM) already demonstrates the underlying discipline needed to apply the same idle-resource control in production. Before full deployment, the priorities are: implement real monitoring/alerting, complete the federated bank-verification integration securely, and validate the AI/ELA container's behavior under sustained load rather than the short test conditions used in CO2 AT2.			

CO2 Reflection and Individual Defense

This section must be individually written. It should show what the student understands from the complete CO2 chain: infrastructure sizing, practical configuration evidence and final operations defense.

Reflection Prompt	Student Response
What is the strongest infrastructure decision in your capstone design?	Choosing a serverless-first architecture over a VM-based one — it directly matches PulsePay's bursty traffic pattern and avoids paying for idle compute, which was made concrete by comparing VM boot time/resource reservation against container startup in CO2 AT2.
Which CO2 concept became clearer after AT1, AT2 and AT3?	AT1 gave me sizing numbers on paper; AT2 made those numbers real by actually configuring a VM, a container, and a cloud VM; AT3 forced me to connect those choices to security, identity, and failure scenarios — the full picture only came together once I had to defend the design as a whole, not just size or configure a piece of it.
Which risk is most serious for your capstone: failure, cost, privacy, scaling or sustainability?	Privacy — PulsePay handles screenshots with partial financial details, so a credential leak or overly broad access policy has a more serious real-world consequence than a temporary outage or cost overrun would.
What evidence would you show during viva to defend your infrastructure design?	The CO2 AT2 evidence showing SSH-key-only auth and the temporary-then-closed port 80 rule on the Azure VM, as concrete proof that least-privilege access thinking was actually practiced, not just described.
What will you improve before final capstone deployment?	Implement real Cloud Monitoring dashboards and Firestore backup automation instead of the currently planned/conceptual versions, and complete the federated bank/NPCI verification integration with a proper access review.

Student-Facing Assessment Criteria

Criteria	Descriptor
Capstone continuity	The answer uses the same capstone title and connects CO1, CO2 AT1 and CO2 AT2 evidence.
Virtualization and lifecycle reasoning	VM, image, snapshot, clone, container or managed-service decisions are technically justified.
Topology and integration clarity	Architecture diagram and SOA/API explanation show clear module interaction.
Identity, access and privacy	Roles, authentication, RBAC, federated access, sensitive data and privacy decisions are defended.
SDDC/cloud operations	Failure response, monitoring, logging, scaling, backup, rollback and resource control are addressed.
Green Computing defense	Right-sizing, retention, API efficiency, idle-resource control and sustainable cloud choices are explained.
Submission quality	GitHub, template completion, evidence references and individual reflection are complete.

GitHub and Google Classroom Submission

Repository Item	Expected Content
README.md	Capstone title, student details, final VM recommendation, deployment topology summary and AT3 evidence index.
Completed AT3 document	This completed editable DOCX or exported PDF if instructed.
Diagram image	Final topology diagram if created separately.
Evidence references	Links or screenshots from CO1, CO2 AT1 and CO2 AT2 as needed.
Green Computing note	Short README section explaining sustainable cloud choices and resource-control decisions.
Google Classroom	Submit the GitHub repository link through GCR.

Student Assurance

I confirm that this case analysis, design reasoning, topology diagram and final defense statement were prepared individually by me for the stated capstone title. I have not uploaded passwords, tokens, private keys or sensitive account information.

Student Name	D.S.THARUNIYA PRIYA
Register Number	192511317
Signature	D.S.Tharuniya Priya
Date	31-07-2026

Faculty Verification

Faculty Name	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Constructive Feedback / Remarks	
Strength Observed	
Improvement Suggested	
Signature / Date	